



Information Security

Lab assignment II

Luis Eduardo Gomez Moran
Submit by 23:59h October 22, 2024

1. Vector Commitment:

1.1. Estructura del código:

El código ha sido separado en 2 archivos (ALICE.py y BOB.py) python para así poder realizar su ejecución en simultaneo y permitir al usuario hacer pruebas pertinentes

Alice.py y Bob.py:

En ambos archivos se realizan operaciones similares y en ocasiones identificas, sin embargo no se realizan al mismo tiempo, ya que en el caso de Bob necesita de datos que genera Alice para poder realizar su propio generador pseudo aleatorio para así comprobar el vector commitment.

El código de Alice contempla las siguientes acciones:

- Solicita por pantalla la longitud del vector m:

```
PS E:\munics\VS CODE\SI LAB\LAB 2> python .\ALICE.py
Define el tamaño de m
```

- Envía el tamaño de m a Bob para que pueda generar r correctamente.

```
PS E:\munics\VS CODE\SI LAB\LAB 2> python .\ALICE.py
Define el tamaño de m
3
m es:
3

PS E:\munics\VS CODE\SI LAB\LAB 2> python .\BOB.py
Bob is connected...
Alice envío: 3
r es:[1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1]
```

- Una vez Bob contesta con el vector r (ver imagen arriba), Alice puede continuar con las siguientes operaciones:
 - Genera b (generado aleatorio)
 - Declara c (sabiendo que c es 3 veces b)
 - Genera la semilla s (generado aleatorio)
 - Genera $G(s)$ (Con input de la semilla)
 - Segmenta $G(s)$ en $G_{nor}(S)$ y $G_r(s)$; donde G_{nor} son las posiciones de $G(s)$ donde el vector r es 0 y $G_r(s)$ son las posiciones de $G(s)$ donde r es 1
 - Genera e (conseguido a partir de hacer $c \oplus G(s)$)

Aquí el resultado:

```
Bob envio: 101000010011101011
q es:
9
x = b'101000010011101011'
l = 18
r es:
[1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1]
b es:
[0, 1, 0]
c es:
[0, 1, 0, 0, 1, 0, 0, 1, 0]
La semilla s es:
[1, 1, 1, 0, 1, 0, 1, 0, 0, 1]
G(s) es :
[0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0]
grS (elementos de G_s donde r es 1):
[0, 1, 1, 0, 0, 0, 0, 1, 0]
gnorS (elementos de G_s donde r es 0):
[0, 0, 1, 0, 0, 1, 0, 0, 0]
e (resultado del XOR): [0, 0, 1, 0, 1, 0, 0, 0, 0]
e a enviar: 001010000
```

- Alice procede a realizar el envío de algunos valores para que análogamente Bob realice las mismas operaciones mencionadas en Alice y compruebe si el vector commitment se cumple :

```
Alice envio: 101011011
El e recibido es:
[1, 0, 1, 0, 1, 1, 0, 1, 1]
Alice envio: 0011110100
Semilla recibida:
[0, 0, 1, 1, 1, 1, 0, 1, 0]
Alice envio: 010
b recibido:
[0, 1, 0]
Alice envio: 101100100
GnorS recibido:
[1, 0, 1, 1, 0, 0, 1, 0, 0]
```

- e (utiliza este valor para comprobar el compromiso vectorial)
- semilla s (se añadió la opción de modificarla para alterar el compromiso)
- vector compromiso b (se añadió la opción de modificarlo para alterar el compromiso)

Bob al recibir e, s y $G_{nor}(S)$ puede realizar:

- Define c_{bob} (Basado en b de Alice)
- Genera $G_{bob}(s)$ (Basado en la semilla de Alice)
- Segmenta $G_{bob}(s)$ en $G_{nor}(S)$ y $G_r(s)$
- Genera e_{bob} (conseguido a partir de hacer $c_{bob} \oplus G_{bob}(s)$)

Aquí podemos observar la ejecución y generación de lo mencionado anteriormente:

```
Alice envio: 101011011
El e recibido es:
[1, 0, 1, 0, 1, 1, 0, 1, 1]
Alice envio: 0011110100
Semilla recibida:
[0, 0, 1, 1, 1, 1, 0, 1, 0, 0]
Alice envio: 010
b recibido:
[0, 1, 0]
Alice envio: 101100100
GnorS recibido:
[1, 0, 1, 1, 0, 0, 1, 0, 0]
c_bob es:
[0, 1, 0, 0, 1, 0, 0, 1, 0]
G(s)_bob :
[1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0]
grS (elementos de G_s donde r es 1):
[1, 1, 1, 0, 0, 1, 0, 0, 1]
gnorS (elementos de G_s donde r es 0):
[1, 0, 1, 1, 0, 0, 1, 0, 0]
e_bob (resultado del XOR):[1, 0, 1, 0, 1, 1, 1, 0, 1, 1]
```

Una vez Bob genera su e_{bob} procede a realizar la comparación del e enviado por Alice con el que obtuvo el operando.

Si son iguales, confirma que el vector commitment esta correcto y le indica a Alice mediante un mensaje confirmando.

De lo contrario le envía un mensaje a Alice indicando que está comprometido.

```
Bob envio: El Compromiso se cumple
Bob envio: La semilla coincide, Alice no hizo trampa
PS E:\munics\VS CODE\SI LAB\LAB 2> |

El compromiso se cumple, Alice es honesta
La semilla coincide, Alice no hizo trampa
```

Explicación funciones:

- on-connect(): Verifica la conexión al MQTT
- on-message(): Verifica si se recibió un mensaje
- Listen(): Mediante la variable chequeo, alternando entre true y false vamos activando y pausando el cliente para escuchar o enviar
- mensaje-a-vec(): Permite transformar los mensajes recibidos en strings de bits a vectores
- g-de-s(s): Toma como entrada el vector semilla, y genera un nuevo vector G_s de tamaño 2^q utilizando la semilla s
- separa-G-s: Esta función toma $G(s)$ y el vector r , la función separa los elementos de la lista G_s en dos listas diferentes, grS y $gnorS$, según los valores del vector r .

$G_r(S)$ contiene los elementos de $G(s)$ en las posiciones donde r tiene un 1.

$G_{nor}(S)$ contiene los elementos de $G(s)$ en las posiciones donde r tiene un 0.

- Calculo-e(): Esta función toma $g_r(S)$ y c realiza una operación bit a bit XOR y arroja el resultado.